ChatGPT

Future-Proof KCU Backtesting Agent

Executive Summary

The most durable design for your KCU agent is a **hybrid system**: TradingView MCP for chart access and replay control, a deterministic Python backtesting engine for candle normalization, resampling, simulation, and audits, an inexpensive Ollama cloud model for **structured tagging only where deterministic rules are insufficient**, Claude Opus for **compressed higher-level reasoning and contradiction review**, a machine-readable memory layer for pattern statistics, and an Obsidian vault for human-readable notes and review. This split is the best match to your requirements because the TradingView MCP project already exposes replay, OHLCV/chart reads, screenshots, and JSON/CLI outputs, but it also explicitly warns that it relies on **undocumented internal TradingView APIs** over Chrome DevTools Protocol and may break when TradingView Desktop changes. The repo also warns that programmatic data consumption may conflict with TradingView's terms and that the tool must not be used for automated trading; that makes it excellent for **personal chart-driven backtesting and annotation**, but not a safe sole dependency for a 90-day scalable historical pipeline. [1]

A low-token design should keep LLMs away from raw candle iteration whenever a deterministic program can do the work. Anthropic's guidance on context engineering emphasizes that context is finite, that long-running agents need **compaction, structured note-taking, and just-in-time retrieval**, and that tools should return **tight, token-efficient outputs**. Claude's prompt-caching and context-editing features exist precisely to reduce repetitive cost and prune stale tool results, while tradingview-mcp itself is designed to return compact outputs and explicitly documents context-saving behaviors such as summary-mode OHLCV and deduplicated Pine drawings. In practice, this means: MCP collects chart state and screenshots, Python performs the heavy lifting, Ollama classifies **candidate windows or ambiguous setups**, and Opus reads only compressed briefs and contradiction logs. [2]

The audit-safe core should be **append-only JSONL event sourcing**. JSON Lines is a good fit because it is designed for one-record-per-line processing, logs, and streaming-friendly structured data. In your case, that enables immutable `candle_review`, `accepted_trade`, and `rejected_setup` events, with later `trade_result` and `post_trade_review` events appended without rewriting prior decisions. That is the cleanest way to enforce anti-lookahead and make human review and memory updates traceable. [3]

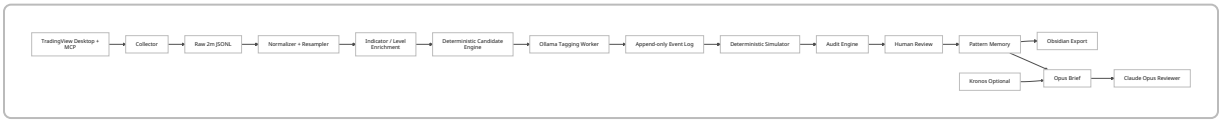
For acquisition, the recommended hierarchy is: **TradingView MCP first for visual replay workflow and screenshot-grounded extraction**, then a licensed historical data provider fallback for scalable range backtests. The fallback decision should be configurable per ticker/provider. Alpaca officially supports historical bars via `/v2/stocks/bars`, but its zero-cost plan only includes limited equities real-time coverage from IEX. Massive's stock aggregate bars are documented in Eastern Time and explicitly state that bars are built from qualifying trades and may be absent when no eligible trades occurred. IBKR supports historical bars with explicit `barSizeSetting`, `whatToShow`, and `useRTH`, but its historical feed is filtered for some off-NBBO trade types and requires subscriptions consistent with live data access. These differences matter for reproducibility and explain why the backtester should track `data_source`, `provider`, and `quality_flags` per run. [4]

For model routing, Ollama's official docs confirm that the local API is served at `http://localhost:11434/api` and the cloud API at `https://ollama.com/api`, with authentication required for cloud use. Ollama's cloud-model docs also state that cloud models are previewed to run larger models with the same local workflow, and the Kimi K2.7 Code cloud model page identifies it as a `cloud` model with `tools`, `thinking`, and a 256K context window. On the Claude side, Claude Code officially supports model pinning and effort-level control, and Opus 4.6 supports `low`, `medium`, `high`, and `max` effort levels. That makes your desired split feasible: **Ollama Kimi K2.7 Code Cloud** for cheaper structured tagging/code-work and **Claude Opus 4.6 medium** for review and synthesis. [5]

Kronos is useful, but only as an **optional context layer after the core system is stable**. The official repo and AAAI paper describe Kronos as a specialized foundation model for financial K-line sequences, pretrained on over 12 billion K-line records from 45 global exchanges, with reported gains in financial forecasting, volatility forecasting, and synthetic K-line generation. That makes it potentially useful for **regime context, volatility expectation, and scenario comparison**, but not as the first-line arbiter of KCU trade validity. Your core system should learn KCU structure first; Kronos should be evaluated later as an ablation: "with Kronos context" vs "without Kronos context." [6]

Architecture and Repository Design

The recommended repository shape is a **single canonical root** with clear separation between KCU canon, connectors, backtesting engine, memory, and Obsidian. This aligns with Claude Code's documented support for a root `CLAUDE.md`, MCP-based tool connections, and hooks that can run commands or prompts at lifecycle events. The root `CLAUDE.md` should define project-wide rules; `backtesting/CLAUDE.md` should define the backtesting contract; hooks should trigger audits, exports, and consistency checks around tool use and file writes. [7]



The future-proof root layout should look like this:

```
KCU_Agent/
  README.md
  CLAUDE.md

  KCU/
    CLAUDE.md
    kcu_framework.md
    studyvault/
    setup_definitions/
      LTP_continuation.md
      Rejection.md
      Reversal.md
      Bounce.md
      Failed_LTP.md
```

```
    Chop.md
  prompts/
    decision_mode.md
    review_mode.md
    memory_update.md

connectors/
  tradingview-mcp/
    README_SETUP.md
    mcp_config.example.json
  obsidian/
    README_SETUP.md
    vault_config.example.yaml

backtesting/
  CLAUDE.md
  config/
    global.yaml
    tickers/
      _template.yaml
      TSLA.yaml
  schemas/
    candle_review.schema.json
    accepted_trade.schema.json
    rejected_setup.schema.json
    trade_result.schema.json
    daily_summary.schema.json
  data/
    raw/{TICKER}/{DATE}/
    normalized/{TICKER}/{DATE}/
    levels/{TICKER}/{DATE}/
  outputs/{TICKER}/{DATE}/
  src/
    collect_from_tradingview.py
    normalize_candles.py
    resample_candles.py
    enrich_indicators.py
    run_backtest_day.py
    run_backtest_range.py
    audit_backtest.py
    update_pattern_memory.py
    export_to_obsidian.py
    query_memory.py
    build_opus_brief.py
    create_ticker_profile.py
    kronos_forecast.py
  reports/
```

```
memory/
  global/
  tickers/{TICKER}/

obsidian_vault/
  Trading Mind Vault/
    00_Index/
    01_KCU_Rules/
    02_Setups/
    03_Tickers/
    04_Daily_Reviews/
    05_Trade_Cards/
    06_Contradictions/
    07_Human_Review/
    08_Screenshots/
    09_Kronos_Context/

models/
  README.md
  ollama_worker_config.yaml
  opus_reviewer_config.yaml
  kronos/

scripts/
  bootstrap_project.sh
  copy_legacy_tsla_files.sh
  validate_project.py
```

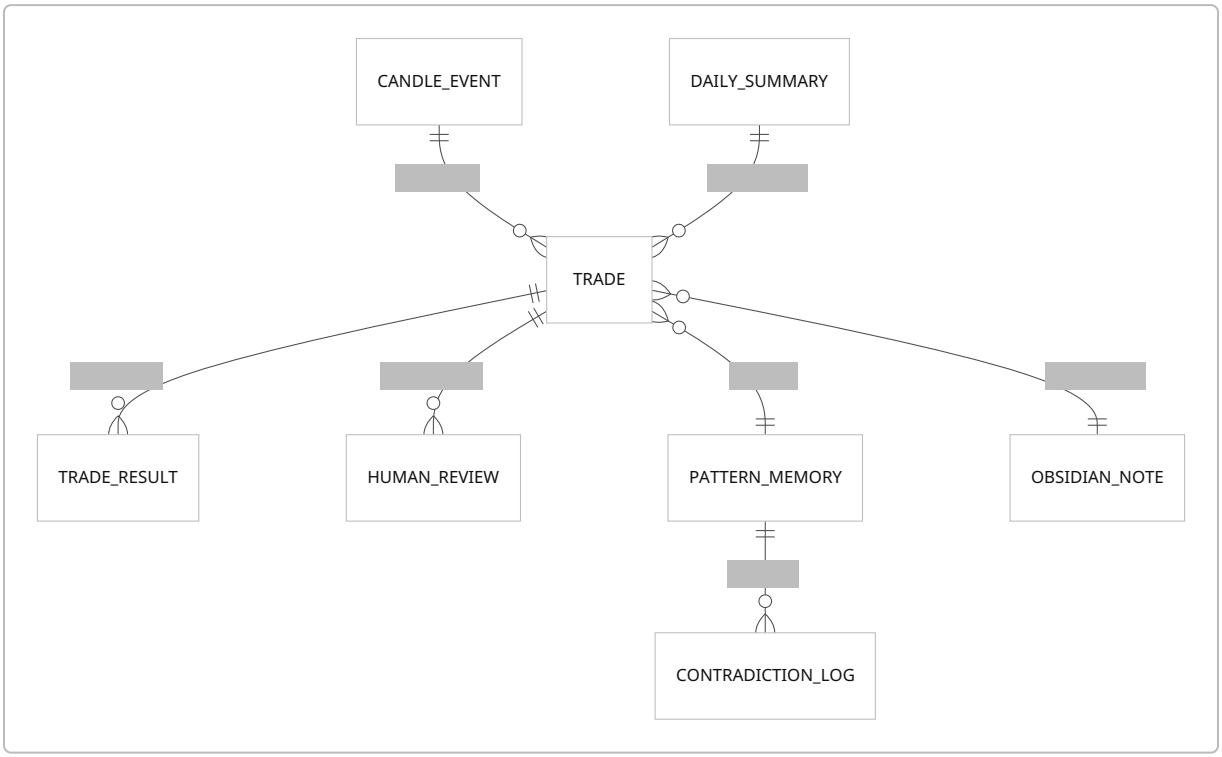
This structure solves the main source of prior messiness: **TSLA-specific logic and KCU canon were mixed into one operational folder**. The cleaner rule is: copy old `TSLA/claude.md`, `kcu_framework.md`, and `studyvault` into `KCU/` as canonical source material, then keep ticker-specific behavior in `backtesting/config/tickers/TSLA.yaml`, `memory/tickers/TSLA/`, and `obsidian_vault/Trading Mind Vault/03_Tickers/TSLA/`. Claude Code's own docs explicitly support root `CLAUDE.md` files as durable session instructions, which is exactly the right mechanism for this project-level control plane. [8]

The connector layer should treat TradingView MCP as a **project-local MCP server** attached through `.mcp.json`. The official repo documents clone + `npm install`, launching TradingView Desktop with `--remote-debugging-port=9222`, and registering the server in `~/.claude/.mcp.json` or a project `.mcp.json`. The same repo also documents useful replay/chart tools such as `replay_start`, `replay_step`, `data_get_ohlcv`, `capture_screenshot`, streaming commands, and a CLI that emits JSON for piping. That gives you a reliable way to keep the collector deterministic and scriptable. [9]

For the Obsidian layer, the official Obsidian docs confirm that note properties are YAML frontmatter, templates can inject reusable note bodies and variables, and the core Search plugin supports fast vault-wide retrieval using operators. That makes Obsidian a strong fit for a **global mind vault**, but it should

remain the **semantic layer**, not the source of truth. The source of truth belongs in JSONL and JSON files under `backtesting/outputs/` and `memory/`. [10]

A simple entity model for the system is:



The model-routing control plane should be explicit and config-driven:

```
models:
  tagging_worker:
    provider: ollama
    endpoint: http://localhost:11434/api
    cloud_endpoint: https://ollama.com/api
    model: kimi-k2.7-code:cloud
    purpose:
      - candidate_window_tagging
      - compact_note_drafts
      - edge_case_classification
  reasoning_reviewer:
    provider: anthropic
    model: claude-opus-4-6
    effort: medium
    purpose:
      - contradiction_review
      - 10d_30d_90d_analysis
      - playbook_synthesis
```

```
  kronos:
    enabled: false
    purpose:
      - optional_regime_context
      - optional_volatility_forecast
```

That configuration is consistent with Ollama's local/cloud API model and Claude Code's model/effort controls. [11]

Data Capture, Schemas, and Deterministic Backtesting

The acquisition strategy should be **hierarchical and provider-aware**. TradingView MCP remains the preferred **visual replay collector** because it can step the chart candle by candle, capture screenshots, stream JSONL, expose chart state and study values, and read TradingView/Pine-generated levels and annotations. But because the repo warns about undocumented APIs, version fragility, and potential terms conflicts for automated data use, it should not be the *only* historical source in a long-running automated pipeline. The backtester should support at least four acquisition modes: `tradingview_mcp`, `alpaca`, `massive_polygon`, and `ibkr`, each writing into the same canonical raw path. [9]

Source	Official capability	Strength	Important caveat	Recommended role
TradingView MCP	Replay, chart navigation, OHLCV, study values, screenshots, JSON CLI, JSONL streaming. [12]	Best for replay-grounded KCU annotation and screenshots	Uses undocumented internal TradingView APIs; may conflict with TradingView ToS for automated collection/non-display use. [13]	Primary visual collector
Alpaca Market Data API	Historical bars endpoint and real-time/historical market data via API. [14]	Good developer ergonomics	Free/basic plan has limited equities real-time coverage from IEX only. [15]	Fallback historical provider
Massive stock aggregates	REST, WebSocket, flat files; aggregate OHLC in ET; no bar when no eligible trades occur. [16]	Strong historical aggregation semantics	Provider behavior depends on qualifying trades and exchange coverage. [17]	Robust historical fallback

Source	Official capability	Strength	Important caveat	Recommended role
IBKR historical bars	`reqHistoricalData` with bar size, RTH toggle, and time-zone controls. [18]	Useful if already subscribed	Historical feed filtered for some trade types; requires relevant market-data subscriptions. [18]	Broker-aligned fallback

The canonical raw path should be:

```
backtesting/data/raw/{TICKER}/{YYYY-MM-DD}/raw_candles_2m.jsonl
```

Normalization then writes:

```
backtesting/data/normalized/{TICKER}/{YYYY-MM-DD}/candles_2m.jsonl
backtesting/data/normalized/{TICKER}/{YYYY-MM-DD}/candles_5m.jsonl
backtesting/data/normalized/{TICKER}/{YYYY-MM-DD}/enriched_candles_2m.jsonl
backtesting/data/normalized/{TICKER}/{YYYY-MM-DD}/enriched_candles_5m.jsonl
backtesting/data/levels/{TICKER}/{YYYY-MM-DD}/levels.json
```

The recommended session model is:

- `04:00–09:29 ET`: **premarket context only**
- `09:30–09:59:59 ET`: **2m primary, 5m confirmation**
- `10:00–16:00 ET`: **5m primary, 2m optional refinement**

That is a design rule derived from your KCU requirements, not an external market-data standard. The collector should still read candles from premarket through close, but acceptance of regular-session trades before 09:30 should remain off unless you later add a distinct premarket strategy.

The deterministic resampling and enrichment path should be:

1. **Normalize timestamp/timezone** to `America/New_York`.
2. **Resample 2m → 5m** using open=first, high=max, low=min, close=last, volume=sum, `gap_flag=true` when a bucket is incomplete or source bars are missing.
3. **Compute levels**:
4. PMH/PML from 04:00–09:29
5. PDH/PDL/PDC from prior regular session
6. ORH/ORL from first 15 minutes by default, ticker-configurable

7. **Compute indicators**:

8. VWAP either from provider study values via MCP or from bar-approximation (unspecified; must be finalized)

9. EMA 8, EMA 21
10. relative volume vs rolling baseline
11. distance to nearest level / obstacle

12. **Annotate market regime**:

13. trend_up, trend_down, range, chop, expansion, exhaustion

The **VWAP implementation method is still unspecified** and must be finalized. If TradingView MCP is the source of truth for chart-visible VWAP, the panel/indicator value should be persisted directly; if fallback providers are used, the system needs a documented bar-based approximation rule.

The core event log should be newline-delimited JSON. The canonical event types and validation constraints are below.

Event type	Mode	Required keys
`premarket_context`	`decision`	`seq,date,time,visible_chart_until,ticker,primary_timeframe,pmh,p`
`candle_review`	`decision`	`seq,date,time,visible_chart_until,ticker,primary_timeframe,confi`
`timeframe_switch`	`decision`	`seq,date,time,ticker,from_primary_timeframe,to_primary_timeframe`
`accepted_trade`	`decision`	`seq,trade_id,date,time,visible_chart_until,ticker,primary_timefr`
`rejected_setup`	`decision`	`seq,date,time,visible_chart_until,ticker,primary_timeframe,atten`
`trade_result`	`outcome`	`seq,trade_id,date,decision_time,result_known_at,visible_chart_un`
`post_trade_review`	`review`	`seq,trade_id,date,classification,review_note,lesson_candidate`
`daily_summary`	`review`	`date,ticker,total_accepted_trades,wins,losses,win_rate,best_setu`
`error_event`	`audit`	`seq,date,time,ticker,error_type,description,corrective_action`

Event type	Mode	Required keys
`human_review`	`review`	`trade_id,human_label,human_note,should_update_memory`

The anti-lookahead rules should be enforced in both the runtime and the audit:

- decisions may only use candles visible at `visible_chart_until`
- decisions are append-only
- outcomes append later
- no decision-mode hindsight phrases
- no 2m-primary decision after 10:00
- if same bar hits both entry and stop/target and no lower timeframe exists, resolve conservatively

A minimal audit pseudocode looks like this:

```
def audit(events):
    assert_jsonl_parseable(events)
    assert_increasing_seq(events)

    switch_seen = False
    accepted = {}

    for e in events:
        if e["event_type"] == "timeframe_switch":
            assert e["time"] == "10:00"
            assert e["from_primary_timeframe"] == "2m"
            assert e["to_primary_timeframe"] == "5m"
            switch_seen = True

        if e["mode"] == "decision":
            t = hhmm_to_minutes(e["time"])
            if 570 <= t < 600:
                assert e["primary_timeframe"] in {"2m", "context"}
            elif t >= 600:
                assert e["primary_timeframe"] == "5m"

            text = serialize_text(e).lower()
            for banned in HINDSIGHT_PHRASES:
                assert banned not in text

        if e["event_type"] == "accepted_trade":
            assert e["entry"] is not None
            assert e["stop"] is not None
            assert e["target_1_5r"] is not None
            assert e["risk"] is not None
            accepted[e["trade_id"]] = hash_trade_definition(e)
```

```
        if e["event_type"] == "trade_result":
            assert e["trade_id"] in accepted

    assert switch_seen
    return True
```

The deterministic simulator should define execution rules explicitly. The safest default policy is:

- **entry** occurs when a stop-entry trigger is crossed after the setup is accepted
- **stop** is beyond patience candle / rejection wick / reversal structure / bounce level
- **target** is `1.5R`
- **obstacle rule**: if clean 1.5R room does not exist at decision time, reject
- **same-bar ambiguity**:
- if lower timeframe exists for disambiguation, use it
- if not, choose the conservative ordering
- **MFE/MAE** are tracked from first fill to exit or session end
- **breakeven management** is configurable; default should be “none” until specified

Your KCU tagging logic should remain partly deterministic and partly model-assisted. The recommended split is:

Setup family	Deterministic screen	LLM tagger role
LTP continuation	level proximity, trend alignment, candle compression candidate, room to target	decide if patience candle is valid when rules are borderline
Rejection	level touch, wick/body rejection candidate, acceptance failure candidate	classify quality of rejection confirmation
Reversal	trend exhaustion + structure shift candidate	decide reversal vs pullback in ambiguous cases
Bounce	level hold + reaction candidate	decide bounce vs rejection vs reversal
Failed setup	missing condition detected	summarize why
Chop	overlap/flat-VWAP/flat-EMA/range metrics	confirm no-trade state if needed

If you insist on “the agent does all tagging,” the safest low-token way is **not** to send full chart screenshots বার-by-bar. Instead, Python should generate a compact per-candle/per-window descriptor, and the Ollama worker should classify that structure. That satisfies your desire for model-driven tagging without paying screenshot-level inference costs on every bar. This is also more aligned with Anthropic’s context-engineering advice: retrieve just enough information at the right time. [19]

Learning, Review, and Knowledge Vault

The self-evolving component should be **case-based learning with human correction**, not rule rewriting from raw outcomes. Anthropic's context-engineering guidance explicitly highlights structured note-taking and persistent memory as a way for agents to maintain coherence across long tasks. In your case, that should become **pattern memory** written outside the immediate context window and updated only after audits and human review. [20]

Every accepted trade should generate:

- `broad_key`
- `specific_key`
- `condition_snapshot`
- `result`
- `decision_quality`
- `human_label` if reviewed
- `lesson_candidate`

Recommended key design:

```
broad_key =
  ticker|setup_type|direction|level_type|trend_state_primary|pc_quality

specific_key =
  ticker|setup_type|direction|time_bucket|primary_timeframe|
  confirmation_timeframe|level_type|trend_state_primary|
  trend_state_confirmation|vwap_state|ema_state|pc_quality|
  volume_state|entry_extension|nearby_obstacle|market_regime|
  multi_timeframe_confirmed
```

Evidence thresholds should be conservative:

Evidence count	Status
1	note
3	weak pattern
5	hypothesis
10	provisional lesson
30	trusted candidate

The crucial label is `decision_quality`, which should stay separate from raw P&L:

Value	Meaning
`valid_win`	rules-valid setup, winning outcome
`valid_loss`	rules-valid setup, losing outcome
`invalid_win`	rules-invalid setup, but made money
`bad_trade`	rules-invalid setup, losing outcome
`unclear`	insufficient evidence

That separation prevents the common failure mode of teaching the system that “all wins are good” and “all losses are bad.”

Contradiction logging should be explicit and human-readable:

```
{
  "pattern_key": "TSLA|LTP_CONTINUATION|LONG|ORH_RECLAIM|BULLISH|CLEAN_PC",
  "previous_outcome": "loss",
  "new_outcome": "win",
  "difference_detected": {
    "entry_extension": ["extended", "not_extended"],
    "multi_timeframe_confirmed": [false, true],
    "nearby_obstacle": ["PDH", "none"]
  },
  "memory_update": "ORH reclaim LTP is conditionally valid when 5m agrees and
there is clean 1.5R room.",
  "confidence": "weak pattern"
}
```

The human review loop is the key quality-control layer. The simplest first implementation is **file-based review**, not a complex web application:

Option	Good for	Cost	Recommendation
`human_review.jsonl` + Obsidian queue	fastest start, most audit-safe	very low	start here
CSV review sheet	quick bulk edits	low	optional export
thin Streamlit/Gradio reviewer	screenshots + approve/reject UX	medium	Phase 2+
full custom web app	rich analytics and collaboration	high	only if needed later

The human review record should look like:

```
{
  "trade_id": "TSLA_2026-06-22_0942_LTP_LONG",
  "human_label": "valid_win",
  "human_note": "Correct read. Clean ORH reclaim with 5m agreement.",
  "should_update_memory": true
}
```

The Obsidian vault should be **global**, with per-ticker districts inside it. Official Obsidian docs support YAML properties, templates, and core search, so the export layer should lean heavily on frontmatter and generated notes. [10]

A recommended trade-card frontmatter is:

```
---
trade_id: TSLA_2026-06-22_0942_LTP_LONG
ticker: TSLA
date: 2026-06-22
time: "09:42"
setup_type: LTP_continuation
direction: long
primary_timeframe: 2m
confirmation_timeframe: 5m
result: win
r_multiple: 1.5
decision_quality: valid_win
broad_key: TSLA|LTP_continuation|long|ORH_reclaim|bullish|clean_pc
specific_key: TSLA|LTP_continuation|long|09:30-10:00|2m|5m|ORH_reclaim|bullish|
bullish|above_vwap|8_over_21|clean_pc|expanding|not_extended|none|trend_up|true
tags:
  - trading/TSLA
  - setup/LTP
  - review/valid_win
---
```

A minimal generated note body should include an auto-generated block only:

```
# TSLA 2026-06-22 09:42 LTP Long

<!-- AUTO-GENERATED START -->
## Setup
- Level: ORH reclaim
- Trend: bullish
- Patience candle: clean
- Multi-timeframe confirmation: yes
```

```
## Trade Plan
- Entry:
- Stop:
- Target 1.5R:

## Outcome
- Result:
- R multiple:
- Decision quality:
<!-- AUTO-GENERATED END -->
```

That pattern is compatible with Obsidian templates and YAML properties, while preserving human annotations outside the protected block. [21]

Suggested Obsidian query patterns:

Question	Query pattern
Which TSLA setups worked 10–11 AM?	`path:"05_Trade_Cards/TSLA" "time: 10:" OR "time: 10"` plus filters on `result` / `decision_quality`
Show valid losses after 10 AM	`path:"05_Trade_Cards/TSLA" "decision_quality: valid_loss"`
Show ORH rejection trades	`path:"05_Trade_Cards/TSLA" "level_type: ORH"`
Show contradictions for TSLA	`path:"06_Contradictions/TSLA"`

Because Obsidian Search is a core plugin with operators and YAML-backed properties, that query model is robust even before adding Dataview or embeddings. [22]

Kronos should sit outside the core review loop at first. Its official materials support its use as a model for K-line data, but the safest integration is a **non-binding sidecar**:

- `kronos/regime_context/{TICKER}/{DATE}.json`
- `kronos/forecasts/{TICKER}/{DATE}/`
- fields such as `forecast_horizon`, `volatility_regime`, `continuation_bias`, `confidence`

The daily backtest can compare KCU-tagged setups against Kronos regime hints later, but Kronos should not alter trade validity until you run specific A/B evaluations. [23]

Rollout, Audits, Risks, and Acceptance

The most important unresolved definitions are below. These should be finalized **before** Phase 1, because ambiguity here will cause inconsistent tagging and poor memory quality.

Missing decision	Current status
Patience Candle exact definition	**Unspecified**
Rejection trigger and confirmation rules	**Partially specified, needs exact trigger**
Reversal trigger and confirmation rules	**Partially specified, needs exact structure break rule**
Bounce setup rule boundary vs rejection/reversal	**Unspecified**
VWAP source method: TradingView indicator vs recomputed bar approximation	**Unspecified**
Opening range duration default: 15m vs ticker-specific	**Partially specified**
Screenshot policy: all accepted/rejected/result events or only key ones	**Unspecified**
Data provider priority in non-MCP runs	**Unspecified**
Same-bar fill/stop ordering policy	**Needs explicit conservative rule**
Breakeven/scale-out policy	**Unspecified; default should be none**
Human review UI	**Unspecified; JSONL + Obsidian recommended first**

The recommended implementation task list is:

Priority	Task	Effort
P0	Bootstrap canonical repository and copy TSLA canon into `KCU/`	3–5 h
P0	Configure TradingView MCP in project `.mcp.json` and verify replay/screenshot/candle reads	2–4 h
P0	Create canonical folder structure, config templates, schema files	4–6 h
P0	Implement raw candle canonicalizer and 2m→5m resampler	4–6 h
P0	Implement level builder (PM/PD/OR) and enrichment pipeline	5–8 h
P0	Implement append-only JSONL event writer and schema validators	5–7 h
P0	Implement one-day deterministic simulator and audit engine	6–10 h

Priority	Task	Effort
P0	Implement basic Obsidian export and human review JSONL template	4–6 h
P1	Implement memory update + contradiction logger	5–8 h
P1	Implement query engine over memory + Obsidian	4–6 h
P1	Implement range runner and missing-data reporting	4–6 h
P2	Add review UI beyond JSONL/Obsidian if needed	8–16 h
P3	Add Kronos sidecar experiment harness	6–12 h

Required script interfaces should look like this:

Script	Purpose	Suggested interface
`collect_from_tradingview.py`	replay-driven collection from MCP	`python backtesting/src/collect_from_tradingview.py --ticker TSLA --date 2026-06-22 --session premarket_to_close`
`normalize_candles.py`	canonicalize schema/ timezone	`python .../normalize_candles.py --ticker TSLA --date 2026-06-22`
`resample_candles.py`	create 5m bars from 2m	`python .../resample_candles.py --ticker TSLA --date 2026-06-22 --from 2m --to 5m`
`enrich_indicators.py`	levels, EMAs, VWAP, regime flags	`python .../enrich_indicators.py --ticker TSLA --date 2026-06-22`
`run_backtest_day.py`	one-day pipeline orchestrator	`python .../run_backtest_day.py --ticker TSLA --date 2026-06-22`
`run_backtest_range.py`	range orchestration	`python .../run_backtest_range.py --ticker TSLA --start 2026-03-01 --end 2026-06-28 --skip-missing`
`audit_backtest.py`	audit outputs	`python .../audit_backtest.py --ticker TSLA --date 2026-06-22`
`update_pattern_memory.py`	pattern and contradiction updates	`python .../update_pattern_memory.py --ticker TSLA --date 2026-06-22`

Script	Purpose	Suggested interface
`export_to_obsidian.py`	note export	`python .../export_to_obsidian.py --ticker TSLA --date 2026-06-22 --vault-path ".../Trading Mind Vault"`
`query_memory.py`	answer playbook questions	`python .../query_memory.py --ticker TSLA "which setups work 10-11"`
`build_opus_brief.py`	compress data for Opus	`python .../build_opus_brief.py --ticker TSLA --range 30d`
`kronos_forecast.py`	optional sidecar	`python .../kronos_forecast.py --ticker TSLA --date 2026-06-22`

The core audit checklist should enforce:

1. JSONL parsing succeeds for every output file.
2. `seq` values strictly increase within each append-only file.
3. premarket has no accepted regular-session trades.
4. before 10:00 all decision events are `2m` or `context`.
5. at or after 10:00 all decision events are `5m`.
6. exactly one `timeframe_switch` at 10:00.
7. no hindsight language in decision-mode events.
8. `accepted_trade` fields are complete.
9. no `trade_result` without prior `accepted_trade`.
10. no mutation of prior decision hashes.
11. memory updates happen only after outcome and, preferably, after human review if available.
12. Obsidian export happens after audit and memory update.

A practical automated test matrix for Phase 0 should include:

Test	Pass criterion
MCP smoke test	`tv_health_check` succeeds; screenshot saved; replay start/step works
File bootstrap test	all canonical folders and stub files created correctly
Schema test	every JSONL line validates against schema definition
Timeframe-switch test	no invalid 2m-primary decisions after 10:00
Anti-lookahead test	banned phrases absent from decision events
Resample test	5m bars match deterministic aggregation from 2m source
Levels test	PM/PD/OR values present in `levels.json`
Simulation test	entry/stop/target and MFE/MAE fields produced for accepted trades

Test	Pass criterion
Review-template test	`human_review.jsonl` template generated
Obsidian export test	trade card and daily review markdown files created
One-day opus-brief test	brief JSON created and parseable

Phase 0 acceptance criteria should be strict:

- one date only
- MCP connected and reproducible
- one full premarket-to-close data pass
- outputs parse cleanly
- audit passes
- human review queue exists
- memory update works on same date
- Obsidian export works
- no 90-day run until Phase 0 and a 3-day sample pass

The phased rollout should be:

Phase	Scope	Goal
Phase 0	1 date	prove end-to-end pipeline
Phase 1	3 dates	trend day, chop day, reversal day
Phase 2	10 dates	stabilize schema, audits, review loop
Phase 3	30 dates	first reliable pattern report
Phase 4	90 dates	playbook-quality statistics and Opus synthesis

The main risk analysis is:

Risk	Why it matters	Mitigation
TradingView MCP API fragility	repo relies on undocumented internal APIs	pin TV Desktop version; fallback provider pipeline; smoke tests before ranges [13]
TradingView ToS / non-display usage risk	repo warns programmatic consumption may conflict with terms	use for personal research only; avoid redistribution/automation; prefer licensed provider for bulk runs [13]
Token blowups	raw-candle LLM loops are expensive and brittle	Python-first pipeline; compact candidate windows; prompt caching; context editing; Opus only on briefs [24]
Label drift	vague setup definitions create noisy memory	finalize PC/rejection/reversal definitions before Phase 1

Risk	Why it matters	Mitigation
Outcome leakage	LLM may rewrite earlier decisions after seeing outcome	append-only event sourcing; trade hashes; review events only
Provider mismatch	provider bars differ in filters/ sessions/time zones	persist provider metadata and normalize aggressively [25]
Over-learning from small samples	rule proposals become noise	evidence thresholds + human gate before trusted rules
Kronos overreach	forecast bias could distort KCU discipline	keep Kronos read-only until explicit A/B evaluation [26]

Master Package Manifest and Key Files

A downloadable manifest file is available here: [master.zip manifest]

The manifest content is:

```
master.zip manifest
KCU_Agent/
README.md
CLAUDE.md
KCU/CLAUDE.md
KCU/kcu_framework.md
KCU/setup_definitions/LTP_continuation.md
KCU/setup_definitions/Rejection.md
KCU/setup_definitions/Reversal.md
KCU/setup_definitions/Bounce.md
KCU/setup_definitions/Failed_LTP.md
KCU/setup_definitions/Chop.md
KCU/prompts/decision_mode.md
KCU/prompts/review_mode.md
KCU/prompts/memory_update.md
connectors/tradingview-mcp/README_SETUP.md
connectors/tradingview-mcp/mcp_config.example.json
connectors/obsidian/README_SETUP.md
connectors/obsidian/vault_config.example.yaml
backtesting/CLAUDE.md
backtesting/config/global.yaml
backtesting/config/tickers/_template.yaml
backtesting/config/tickers/TSLA.yaml
backtesting/schemas/candle_review.schema.json
backtesting/schemas/accepted_trade.schema.json
backtesting/schemas/rejected_setup.schema.json
```

```
backtesting/schemas/trade_result.schema.json
backtesting/schemas/daily_summary.schema.json
backtesting/src/collect_from_tradingview.py
backtesting/src/normalize_candles.py
backtesting/src/resample_candles.py
backtesting/src/enrich_indicators.py
backtesting/src/run_backtest_day.py
backtesting/src/run_backtest_range.py
backtesting/src/audit_backtest.py
backtesting/src/update_pattern_memory.py
backtesting/src/export_to_obsidian.py
backtesting/src/query_memory.py
backtesting/src/build_opus_brief.py
backtesting/src/create_ticker_profile.py
backtesting/src/kronos_forecast.py
backtesting/reports/.gitkeep
memory/global/cross_ticker_patterns.json
memory/global/kcu_universal_lessons.md
memory/tickers/TSLA/pattern_memory.json
memory/tickers/TSLA/contradiction_log.jsonl
memory/tickers/TSLA/proposed_lessons.md
memory/tickers/TSLA/trusted_rules.md
obsidian_vault/Trading Mind Vault/00_Index/Master Dashboard.md
obsidian_vault/Trading Mind Vault/01_KCU_Rules/KCU Core Framework.md
obsidian_vault/Trading Mind Vault/02_Setups/LTP Continuation.md
obsidian_vault/Trading Mind Vault/02_Setups/Rejection Trade.md
obsidian_vault/Trading Mind Vault/02_Setups/Reversal Trade.md
obsidian_vault/Trading Mind Vault/02_Setups/Bounce Trade.md
obsidian_vault/Trading Mind Vault/02_Setups/Failed LTP.md
obsidian_vault/Trading Mind Vault/02_Setups/Chop No Trade.md
obsidian_vault/Trading Mind Vault/03_Tickers/TSLA/TSLA Playbook.md
obsidian_vault/Trading Mind Vault/04_Daily_Reviews/TSLA/.gitkeep
obsidian_vault/Trading Mind Vault/05_Trade_Cards/TSLA/.gitkeep
obsidian_vault/Trading Mind Vault/06_Contradictions/TSLA/.gitkeep
obsidian_vault/Trading Mind Vault/07_Human_Review/TSLA/Review Queue.md
obsidian_vault/Trading Mind Vault/08_Screenshots/TSLA/.gitkeep
obsidian_vault/Trading Mind Vault/09_Kronos_Context/TSLA/.gitkeep
models/README.md
models/ollama_worker_config.yaml
models/opus_reviewer_config.yaml
models/kronos/README_SETUP.md
scripts/bootstrap_project.sh
scripts/copy_legacy_tsla_files.sh
scripts/validate_project.py
```

Below are the proposed **exact reference contents** for the key files you asked for.

`CLAUDE.md`

```
# KCU Agent Master Instructions

## Mission

This repository builds a future-proof, low-token, audit-safe KCU backtesting
agent.

Primary goals:
- Learn KCU setup behavior from historical chart data.
- Support multiple tickers with one shared engine.
- Keep market-data collection, deterministic backtesting, tagging, review,
memory, and note export separate.
- Never auto-trade.
- Never use future information in decision events.

## Source of truth hierarchy

1. JSONL / JSON outputs under `backtesting/outputs/`
2. Machine memory under `memory/`
3. Human review under `backtesting/outputs/.../human_review.jsonl`
4. Obsidian notes under `obsidian_vault/`
5. Optional Kronos context under `models/kronos/`

Obsidian is not the source of truth.
Kronos is not the source of truth.

## Canonical rules

Always read:
- `KCU/CLAUDE.md`
- `KCU/kcu_framework.md`
- `backtesting/CLAUDE.md`

Use `KCU/` as the canonical rules layer.
Do not modify copied legacy files in the old TSLA folder.

## Non-negotiable rules

- No future candles in decision events.
- No rewriting locked decisions.
- No 2m-primary decision events after 10:00 ET.
- No automatic promotion of proposed lessons into trusted rules.
- No live trading or trade execution.
- No assumption that TradingView MCP is always stable.
- No dependence on a single data source.
```

```
- No full-range 90-day run until Phase 0 through Phase 2 pass.

## Data-source policy

Preferred visual collector:
- TradingView Desktop through tradingview-mcp.

Fallback historical providers:
- provider configured in `backtesting/config/global.yaml`
- per-ticker overrides allowed in `backtesting/config/tickers/{TICKER}.yaml`

Canonical raw path:
- `backtesting/data/raw/{TICKER}/{DATE}/raw_candles_2m.jsonl`

Canonical output path:
- `backtesting/outputs/{TICKER}/{DATE}/`

## Project structure contract

- `KCU/` = canonical setup logic and study material
- `connectors/` = tradingview-mcp and Obsidian setup docs
- `backtesting/` = data, schemas, scripts, reports
- `memory/` = machine-readable learning state
- `obsidian_vault/` = human-readable linked notes
- `models/` = model routing and optional Kronos sidecar
- `scripts/` = bootstrap and validation helpers

## Model routing

Python deterministic engine:
- normalization
- resampling
- indicators
- levels
- candidate detection
- entry/stop/target simulation
- audit checks
- memory aggregation

Ollama tagging worker:
- structured tagging only
- edge-case classification only
- short note drafting only

Claude Opus reviewer:
- contradiction review
- 10d / 30d / 90d analysis
- playbook synthesis
```

```
- proposed rule reviews

Kronos:
- optional later context
- no rule override
- no decision override

## Human review policy

The system must support human correction.

Every accepted trade can be reviewed with:
- valid_win
- valid_loss
- invalid_win
- bad_trade
- not_a_setup
- missed_setup
- wrong_setup_type
- wrong_entry
- wrong_stop
- wrong_context

Human review should update machine memory.
Human review should not silently rewrite historical events.

## Low-token policy

Use compact structured data first.

Never send full raw-day candle text to a model if Python can compute the result.

Preferred order:
1. collect structured candles
2. normalize
3. enrich
4. detect structured candidate events
5. use a model only on candidate windows or compressed summaries
6. export and review

## Phase policy

Phase 0:
- one date
- one ticker
- prove end-to-end

Phase 1:
```

```
- three dates
- trend / chop / reversal coverage

Phase 2:
- ten dates
- stabilize schemas and reviews

Phase 3:
- thirty dates
- first reliable pattern stats

Phase 4:
- ninety dates
- final playbook-quality memory

Kronos only after Phase 2 is stable.

## Migration policy

Copy old TSLA materials into:
- `KCU/CLAUDE.md`
- `KCU/kcu_framework.md`
- `KCU/studyvault/`

Do not move or destroy the old source until the new repository is validated.

## Final objective

The final system should answer questions like:
- Which TSLA setups work best from 10–11 AM?
- Which setups fail after 11 AM?
- What changed when the same setup failed before and worked later?
- Which patterns deserve trusted-rule candidate status?

Answers must include:
- sample size
- win rate
- avg R
- decision-quality split
- confidence level
- example trade IDs
- conditions and failure conditions
```

`backtesting/CLAUDE.md`

```
# Backtesting Instructions

## Mission

This file governs all backtesting flows under `backtesting/`.

Each backtest must produce:
- canonical raw data
- normalized data
- enriched data
- append-only event logs
- deterministic outcomes
- audit report
- optional memory update
- optional Obsidian export
- optional Opus brief

## Session scope

Collect candles from:
- 04:00 to 16:00 America/New_York

Use:
- 04:00–09:29 as context only
- 09:30–09:59:59 as 2m-primary decision window
- 10:00–16:00 as 5m-primary decision window

## Setup families

Supported setup types:
- LTP_continuation
- rejection
- reversal
- bounce
- failed_LTP
- failed_rejection
- failed_reversal
- failed_bounce
- chop
- no_trade

## Timeframe rules

Hard rule:
- Before 10:00 ET, primary timeframe = 2m
- At/after 10:00 ET, primary timeframe = 5m
- At exactly 10:00 ET, emit one `timeframe_switch` event
```

```
Use 5m as the structure lens.
Use 2m as the execution lens.

If 2m and 5m disagree:
- 5m decides direction and structure
- 2m may refine timing only

No 2m-primary decision event may exist after 10:00 ET.

## Anti-lookahead

At candle N:
- only use information visible through candle N
- write the decision before advancing
- never rewrite earlier decisions after seeing future candles

Decision events:
- premarket_context
- candle_review
- timeframe_switch
- accepted_trade
- rejected_setup

Outcome / review events:
- trade_result
- post_trade_review
- daily_summary
- error_event
- human_review

Never merge hindsight into decision events.

Banned hindsight phrases in decision events:
- would have
- ended up
- eventually
- later confirmed
- worked because
- failed because
- after seeing
- in hindsight
- target hit
- stop hit

## Output contract

Required output folder:
```

```
- `backtesting/outputs/{TICKER}/{DATE}/`

Required files:
- `mcp_candle_collection.jsonl` if TradingView MCP is used
- `events_append_only.jsonl`
- `candle_log.jsonl`
- `accepted_trades.jsonl`
- `rejected_setups.jsonl`
- `trade_results.jsonl`
- `human_review.jsonl`
- `daily_summary.json`
- `audit_report.md`
- `key_screenshot_requests.jsonl`
- `screenshots/`

## Canonical event rules

Every record must contain:
- `seq`
- `event_type`
- `date`
- `ticker`

Decision events must contain:
- `time`
- `visible_chart_until`
- `primary_timeframe`

Accepted trades must contain:
- `setup_type`
- `direction`
- `entry`
- `stop`
- `target_1_5r`
- `risk`
- `reason_codes`

Trade results must contain:
- `trade_id`
- `decision_time`
- `result_known_at`
- `result`
- `r_multiple`
- `max_favorable_r`
- `max_adverse_r`
- `valid_loss_or_bad_trade`

## Deterministic execution defaults
```

```
Default target:
- 1.5R

Reject instead of accept when:
- no clean room to target
- major obstacle blocks 1.5R
- timeframe conflict invalidates setup
- chop invalidates setup
- partial LTP only

If same bar is ambiguous:
- use lower timeframe if available
- otherwise choose the conservative outcome

## Memory update rules

Every completed trade should produce:
- broad_key
- specific_key
- condition_snapshot
- result
- decision_quality
- lesson_candidate

Evidence thresholds:
- 1 = note
- 3 = weak pattern
- 5 = hypothesis
- 10 = provisional lesson
- 30 = trusted candidate

Do not auto-update `trusted_rules.md`.

## Obsidian export rules

Obsidian notes are optional output but must preserve human edits.

Only update content inside:
- `<!-- AUTO-GENERATED START -->`
- `<!-- AUTO-GENERATED END -->`

Generate:
- trade cards
- daily review
- contradiction notes
- ticker dashboard
```

```
## Audit rules

Every date must be audited for:
- JSONL parseability
- increasing seq
- timeframe switch at 10:00
- no 2m-primary decisions after 10:00
- no hindsight language in decision events
- accepted trades complete
- trade results occur after trade accepts
- memory updates happen after outcomes

If audit fails:
- emit `error_event`
- mark day failed
- do not promote lessons
- do not run large batch by default

## Review loop

Create `human_review.jsonl` template for every day.

Human review may mark:
- valid_win
- valid_loss
- invalid_win
- bad_trade
- not_a_setup
- missed_setup
- wrong_setup_type
- wrong_entry
- wrong_stop
- wrong_context

Human review informs memory.

## Allowed automation

Allowed:
- hooks that run audits after file generation
- hooks that prepare review queues
- hooks that export markdown after audit passes

Disallowed:
- hidden mutation of historical logs
- automatic trusted-rule promotion
- automatic live-trading actions
```

`backtesting/config/global.yaml`

```yaml
project:
  name: kcu_backtesting_agent
  timezone: America/New_York
  default_ticker: TSLA

session:
  premarket_start: "04:00"
  regular_start: "09:30"
  switch_to_5m: "10:00"
  regular_end: "16:00"

timeframe_rules:
  premarket:
    primary_timeframe: context
    confirmation_timeframe: none
    trades_allowed: false
  open_window:
    start: "09:30"
    end: "09:59:59"
    primary_timeframe: "2m"
    confirmation_timeframe: "5m"
  regular_after_open:
    start: "10:00"
    end: "16:00"
    primary_timeframe: "5m"
    confirmation_timeframe: "2m"

risk:
  default_rr: 1.5
  breakeven_policy: none
  same_bar_policy: conservative_if_ambiguous

data:
  provider: tradingview_mcp
  canonical_raw_path: "backtesting/data/raw/{TICKER}/{DATE}/
raw_candles_2m.jsonl"
  canonical_output_path: "backtesting/outputs/{TICKER}/{DATE}/"
  fallback_providers:
    - alpaca
    - massive_polygon
    - ibkr
  persist_provider_metadata: true

setups:
  enabled:
```

```
    - LTP_continuation
    - rejection
    - reversal
    - bounce
    - failed_LTP
    - failed_rejection
    - failed_reversal
    - failed_bounce
    - chop
    - no_trade

learning:
  thresholds:
    note: 1
    weak_pattern: 3
    hypothesis: 5
    provisional_lesson: 10
    trusted_candidate: 30
  require_human_review_for_trusted_candidates: true

audit:
  enforce_no_lookahead: true
  enforce_timeframe_switch: true
  enforce_append_only_events: true
  reject_2m_primary_after_10: true
  fail_on_invalid_jsonl: true
  banned_hindsight_phrases:
    - "would have"
    - "ended up"
    - "eventually"
    - "later confirmed"
    - "worked because"
    - "failed because"
    - "after seeing"
    - "in hindsight"
    - "target hit"
    - "stop hit"

obsidian:
  enabled: true
  vault_path: "/ABSOLUTE/PATH/TO/Trading Mind Vault"
  export_root_fallback: "obsidian_vault/Trading Mind Vault"

models:
  tagging_worker:
    provider: ollama
    endpoint: "http://localhost:11434/api"
    cloud_endpoint: "https://ollama.com/api"
```

```
    model: "kimi-k2.7-code:cloud"
  reasoning_reviewer:
    provider: anthropic
    model: "claude-opus-4-6"
    effort: medium
  kronos:
    enabled: false

phases:
  current: phase0
  allow_range_runs: false
```

`backtesting/config/tickers/_template.yaml`

```
ticker: "{TICKER}"
asset_type: equity
exchange: "{EXCHANGE}"

profile:
  personality: unknown
  notes: []

levels:
  required:
    - PMH
    - PML
    - PDH
    - PDL
    - PDC
    - ORH
    - ORL
    - VWAP
  optional:
    - AVWAP
    - whole_dollar
    - half_dollar
    - prior_intraday_support
    - prior_intraday_resistance

time_windows:
  "04:00-09:29":
    trades_allowed: false
    role: context_only
  "09:30-10:00":
    primary_timeframe: "2m"
    confirmation_timeframe: "5m"
```

```
    min_grade_to_accept: "A"
  "10:00-11:00":
    primary_timeframe: "5m"
    confirmation_timeframe: "2m"
    min_grade_to_accept: "A"
  "11:00-14:00":
    primary_timeframe: "5m"
    confirmation_timeframe: "2m"
    min_grade_to_accept: "A+"
  "14:00-16:00":
    primary_timeframe: "5m"
    confirmation_timeframe: "2m"
    min_grade_to_accept: "A+"

setups:
  preferred:
    - LTP_continuation
    - rejection
    - reversal
    - bounce
  filters:
    require_room_to_target: true
    confirm_multi_timeframe_when_possible: true
    reject_partial_ltp: true

parameters:
  opening_range_minutes: 15
  max_pc_range_pct_of_price: 0.005
  min_relative_volume: 1.0
  avoid_if_entry_extension_pct: 0.004
  strict_after_11: true

memory:
  path: "memory/tickers/{TICKER}/"

obsidian:
  dashboard: "03_Tickers/{TICKER}/{TICKER} Dashboard.md"
  playbook: "03_Tickers/{TICKER}/{TICKER} Playbook.md"

data:
  preferred_provider: tradingview_mcp
  fallback_providers:
    - alpaca
    - massive_polygon
    - ibkr
```

`backtesting/src/run_backtest_day.py`

```python
from __future__ import annotations

import argparse
import json
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable

# NOTE:
# This is a Phase 0 stub. It creates the folder structure, validates inputs,
# and writes placeholder outputs that downstream scripts can consume.
# Replace TODO sections with production logic.

ROOT = Path(__file__).resolve().parents[1]


@dataclass
class BacktestPaths:
    root: Path
    raw_dir: Path
    normalized_dir: Path
    levels_dir: Path
    output_dir: Path


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run one KCU backtest day.")
    parser.add_argument("--ticker", required=True, help="Ticker symbol, e.g.
TSLA")
    parser.add_argument("--date", required=True, help="Trading date in YYYY-MM-
DD")
    parser.add_argument(
        "--config",
        default=str(ROOT / "config" / "global.yaml"),
        help="Path to global config YAML",
    )
    parser.add_argument(
        "--ticker-config",
        default=None,
        help="Path to ticker YAML; defaults to config/tickers/{TICKER}.yaml",
    )
    parser.add_argument(
        "--raw-file",
        default=None,
        help="Optional explicit raw 2m JSONL input file path",
    )
```

```
    parser.add_argument(
        "--skip-memory",
        action="store_true",
        help="Skip pattern-memory update",
    )
    parser.add_argument(
        "--skip-obsidian",
        action="store_true",
        help="Skip Obsidian export",
    )
    return parser.parse_args()


def ensure_paths(ticker: str, date: str) -> BacktestPaths:
    raw_dir = ROOT / "data" / "raw" / ticker / date
    normalized_dir = ROOT / "data" / "normalized" / ticker / date
    levels_dir = ROOT / "data" / "levels" / ticker / date
    output_dir = ROOT / "outputs" / ticker / date
    for p in (raw_dir, normalized_dir, levels_dir, output_dir, output_dir /
"screenshots"):
        p.mkdir(parents=True, exist_ok=True)
    return BacktestPaths(ROOT, raw_dir, normalized_dir, levels_dir, output_dir)


def read_jsonl(path: Path) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    with path.open("r", encoding="utf-8") as fh:
        for line_no, line in enumerate(fh, start=1):
            line = line.strip()
            if not line:
                continue
            try:
                rows.append(json.loads(line))
            except json.JSONDecodeError as exc:
                raise ValueError(f"Invalid JSONL in {path} at line {line_no}:
{exc}") from exc
    return rows


def write_jsonl(path: Path, rows: Iterable[dict[str, Any]]) -> None:
    with path.open("w", encoding="utf-8") as fh:
        for row in rows:
            fh.write(json.dumps(row, ensure_ascii=False) + "\n")


def write_json(path: Path, payload: dict[str, Any]) -> None:
    path.write_text(json.dumps(payload, indent=2, ensure_ascii=False) + "\n",
encoding="utf-8")
```

```
def bootstrap_outputs(output_dir: Path, ticker: str, date: str) -> None:
    empty_jsonl_files = [
        "events_append_only.jsonl",
        "candle_log.jsonl",
        "accepted_trades.jsonl",
        "rejected_setups.jsonl",
        "trade_results.jsonl",
        "key_screenshot_requests.jsonl",
        "human_review.jsonl",
    ]
    for fname in empty_jsonl_files:
        fpath = output_dir / fname
        if not fpath.exists():
            fpath.write_text("", encoding="utf-8")

    summary = {
        "event_type": "daily_summary",
        "date": date,
        "ticker": ticker,
        "status": "initialized",
        "notes": [
            "Phase 0 stub created output files.",
            "Populate with production backtest results after implementing
collectors, taggers, and simulator.",
        ],
    }
    write_json(output_dir / "daily_summary.json", summary)
    (output_dir / "audit_report.md").write_text(
        "# Audit Report\n\nStatus: not yet run.\n",
        encoding="utf-8",
    )


def locate_raw_file(args: argparse.Namespace, paths: BacktestPaths) -> Path:
    if args.raw_file:
        candidate = Path(args.raw_file)
        if candidate.exists():
            return candidate
        raise FileNotFoundError(f"--raw-file does not exist: {candidate}")

    canonical = paths.raw_dir / "raw_candles_2m.jsonl"
    if canonical.exists():
        return canonical

    legacy_candidates = [
        ROOT.parent / "setups" / "tsla" / "outputs" / args.date /
```

```
"raw_candles_2m.jsonl",
        ROOT / "raw_data" / args.ticker / args.date / "raw_candles_2m.jsonl",
    ]
    for candidate in legacy_candidates:
        if candidate.exists():
            canonical.write_text(candidate.read_text(encoding="utf-8"),
encoding="utf-8")
            return canonical

    raise FileNotFoundError(
        "Raw 2m candle file not found in canonical or legacy locations. "
        "Populate backtesting/data/raw/{ticker}/{date}/raw_candles_2m.jsonl
first."
    )


def normalize_and_copy_raw(raw_file: Path, normalized_out: Path) ->
list[dict[str, Any]]:
    rows = read_jsonl(raw_file)
    # TODO: enforce timezone normalization and canonical field names.
    write_jsonl(normalized_out, rows)
    return rows


def resample_to_5m(rows_2m: list[dict[str, Any]]) -> list[dict[str, Any]]:
    # TODO: replace with deterministic resampling logic.
    # Placeholder behavior: copy every row marked as placeholder_5m=false.
    rows_5m: list[dict[str, Any]] = []
    for row in rows_2m:
        out = dict(row)
        out["timeframe"] = "5m_placeholder"
        rows_5m.append(out)
    return rows_5m


def enrich_rows(rows: list[dict[str, Any]], timeframe: str) -> list[dict[str,
Any]]:
    enriched: list[dict[str, Any]] = []
    for row in rows:
        item = dict(row)
        item["timeframe"] = timeframe
        item.setdefault("vwap", None)
        item.setdefault("ema_8", None)
        item.setdefault("ema_21", None)
        item.setdefault("pdh", None)
        item.setdefault("pdl", None)
        item.setdefault("pdc", None)
        item.setdefault("pmh", None)
```

```
        item.setdefault("pml", None)
        item.setdefault("orh", None)
        item.setdefault("orl", None)
        enriched.append(item)
    return enriched


def record_stub_event(output_dir: Path, ticker: str, date: str, raw_count: int)
-> None:
    rows = [
        {
            "seq": 1,
            "mode": "decision",
            "event_type": "premarket_context",
            "date": date,
            "time": "04:00",
            "visible_chart_until": f"{date}T04:00:00-04:00",
            "ticker": ticker,
            "primary_timeframe": "context",
            "message": f"Initialized Phase 0 stub with {raw_count} raw bars.",
        }
    ]
    write_jsonl(output_dir / "events_append_only.jsonl", rows)
    write_jsonl(output_dir / "candle_log.jsonl", rows)


def main() -> int:
    args = parse_args()
    ticker = args.ticker.upper()
    ticker_config = Path(args.ticker_config) if args.ticker_config else ROOT /
"config" / "tickers" / f"{ticker}.yaml"

    if not Path(args.config).exists():
        print(f"Missing global config: {args.config}", file=sys.stderr)
        return 2
    if not ticker_config.exists():
        print(f"Missing ticker config: {ticker_config}", file=sys.stderr)
        return 2

    paths = ensure_paths(ticker, args.date)
    raw_file = locate_raw_file(args, paths)

    if paths.raw_dir / "raw_candles_2m.jsonl" != raw_file:
        (paths.raw_dir /
"raw_candles_2m.jsonl").write_text(raw_file.read_text(encoding="utf-8"),
encoding="utf-8")

    bootstrap_outputs(paths.output_dir, ticker, args.date)
```

```
    rows_2m = normalize_and_copy_raw(raw_file, paths.normalized_dir /
"candles_2m.jsonl")
    rows_5m = resample_to_5m(rows_2m)
    write_jsonl(paths.normalized_dir / "candles_5m.jsonl", rows_5m)

    write_jsonl(paths.normalized_dir / "enriched_candles_2m.jsonl",
enrich_rows(rows_2m, "2m"))
    write_jsonl(paths.normalized_dir / "enriched_candles_5m.jsonl",
enrich_rows(rows_5m, "5m"))

    levels = {
        "ticker": ticker,
        "date": args.date,
        "status": "placeholder",
        "notes": "Replace with deterministic PM/PD/OR/VWAP level builder.",
    }
    write_json(paths.levels_dir / "levels.json", levels)
    record_stub_event(paths.output_dir, ticker, args.date, len(rows_2m))

    print(json.dumps(
        {
            "status": "ok",
            "ticker": ticker,
            "date": args.date,
            "raw_file": str(raw_file),
            "normalized_2m": str(paths.normalized_dir / "candles_2m.jsonl"),
            "normalized_5m": str(paths.normalized_dir / "candles_5m.jsonl"),
            "output_dir": str(paths.output_dir),
        },
        indent=2,
    ))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`backtesting/src/audit_backtest.py`

```
from __future__ import annotations

import argparse
import json
import re
from pathlib import Path
```

```
from typing import Any

ROOT = Path(__file__).resolve().parents[1]
BANNED_HINTSIGHT = [
    "would have",
    "ended up",
    "eventually",
    "later confirmed",
    "worked because",
    "failed because",
    "after seeing",
    "in hindsight",
    "target hit",
    "stop hit",
]


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Audit KCU backtest outputs.")
    parser.add_argument("--ticker", required=True)
    parser.add_argument("--date", required=True)
    return parser.parse_args()


def read_jsonl(path: Path) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    if not path.exists():
        return rows
    with path.open("r", encoding="utf-8") as fh:
        for line_no, line in enumerate(fh, start=1):
            line = line.strip()
            if not line:
                continue
            rows.append(json.loads(line))
    return rows


def parse_hhmm(value: str) -> int:
    match = re.match(r"^(\d{2}):(\d{2})", value)
    if not match:
        return -1
    hh, mm = match.groups()
    return int(hh) * 60 + int(mm)


def find_text(obj: Any) -> str:
    if isinstance(obj, dict):
        return " ".join(find_text(v) for v in obj.values())
```

```
    if isinstance(obj, list):
        return " ".join(find_text(v) for v in obj)
    if isinstance(obj, str):
        return obj
    return ""


def audit_rows(events: list[dict[str, Any]]) -> list[str]:
    errors: list[str] = []
    last_seq = -1
    has_switch = False

    for row in events:
        seq = int(row.get("seq", -1))
        if seq <= last_seq:
            errors.append(f"Non-increasing seq at event {seq}.")
        last_seq = seq

        event_type = row.get("event_type", "")
        mode = row.get("mode", "")
        time_value = row.get("time") or row.get("decision_time") or ""
        minute_of_day = parse_hhmm(time_value)

        if event_type == "timeframe_switch" and time_value.startswith("10:00"):
            has_switch = True

        if mode == "decision":
            tf = row.get("primary_timeframe")
            if 570 <= minute_of_day < 600 and tf not in {"2m", "context"}:

errors.append(f"Decision event before 10:00 must use 2m/context, got {tf} at
seq {seq}.")
            if minute_of_day >= 600 and tf == "2m":
                errors.append(f"Decision event after 10:00 must not use 2m at
seq {seq}.")
            text = find_text(row).lower()
            for phrase in BANNED_HINTSIGHT:
                if phrase in text:
                    errors.append(f"Hindsight phrase '{phrase}' found in seq
{seq}.")
                    break

        if event_type == "accepted_trade":
            required = ["entry", "stop", "target_1_5r", "risk", "reason_codes"]
            for key in required:
                if row.get(key) in (None, "", []):
                    errors.append(f"accepted_trade seq {seq} missing {key}.")
```

```
    if not has_switch:
        errors.append("Missing timeframe_switch event at 10:00.")
    return errors


def main() -> int:
    args = parse_args()
    output_dir = ROOT / "outputs" / args.ticker.upper() / args.date
    events_path = output_dir / "events_append_only.jsonl"
    report_path = output_dir / "audit_report.md"

    events = read_jsonl(events_path)
    errors = audit_rows(events)

    lines = ["# Audit Report", "", f"Ticker: {args.ticker.upper()}", f"Date:
{args.date}", ""]
    if errors:
        lines.append("Status: FAIL")
        lines.append("")
        lines.append("## Errors")
        lines.extend(f"- {e}" for e in errors)
    else:
        lines.append("Status: PASS")
        lines.append("")
        lines.append("No audit errors found.")

    report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
    print(json.dumps({"status": "fail" if errors else "pass", "error_count":
len(errors)}, indent=2))
    return 1 if errors else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`backtesting/src/update_pattern_memory.py`

```
from __future__ import annotations

import argparse
import json
from collections import defaultdict
from pathlib import Path
from typing import Any

ROOT = Path(__file__).resolve().parents[1]
```

```
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Update pattern memory from
backtest outputs.")
    parser.add_argument("--ticker", required=True)
    parser.add_argument("--date", required=True)
    return parser.parse_args()


def read_jsonl(path: Path) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    if not path.exists():
        return rows
    with path.open("r", encoding="utf-8") as fh:
        for line in fh:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows


def read_json(path: Path, default: Any) -> Any:
    if not path.exists():
        return default
    return json.loads(path.read_text(encoding="utf-8"))


def broad_key(trade: dict[str, Any]) -> str:
    return "|".join(
        [
            trade.get("ticker", ""),
            trade.get("setup_type", ""),
            trade.get("direction", ""),
            trade.get("level_type", ""),
            trade.get("trend_context_primary", ""),
            trade.get("pc_quality", ""),
        ]
    )


def specific_key(trade: dict[str, Any]) -> str:
    return "|".join(
        [
            trade.get("ticker", ""),
            trade.get("setup_type", ""),
            trade.get("direction", ""),
            trade.get("time_bucket", ""),
```

```
            trade.get("primary_timeframe", ""),
            trade.get("confirmation_timeframe", ""),
            trade.get("level_type", ""),
            trade.get("trend_context_primary", ""),
            trade.get("trend_context_confirmation", ""),
            trade.get("vwap_state", ""),
            trade.get("ema_state", ""),
            trade.get("pc_quality", ""),
            trade.get("volume_state", ""),
            trade.get("entry_extension", ""),
            trade.get("nearby_obstacle", ""),
            trade.get("market_regime", ""),
            str(trade.get("multi_timeframe_confirmed", False)).lower(),
        ]
    )


def bucket_time(hhmm: str) -> str:
    if hhmm < "10:00":
        return "09:30-10:00"
    if hhmm < "11:00":
        return "10:00-11:00"
    if hhmm < "14:00":
        return "11:00-14:00"
    return "14:00-16:00"


def main() -> int:
    args = parse_args()
    ticker = args.ticker.upper()
    out_dir = ROOT / "outputs" / ticker / args.date
    mem_dir = ROOT.parent / "memory" / "tickers" / ticker
    mem_dir.mkdir(parents=True, exist_ok=True)

    accepted = read_jsonl(out_dir / "accepted_trades.jsonl")
    results = {row.get("trade_id"): row for row in read_jsonl(out_dir /
"trade_results.jsonl")}
    memory_path = mem_dir / "pattern_memory.json"
    contradiction_path = mem_dir / "contradiction_log.jsonl"

    memory = read_json(memory_path, {})
    contradictions: list[dict[str, Any]] = []

    for trade in accepted:
        trade_id = trade.get("trade_id")
        result = results.get(trade_id, {})
        trade["time_bucket"] = bucket_time(str(trade.get("time", "")))
        bkey = broad_key(trade)
```

```
        skey = specific_key(trade)
        classification = result.get("valid_loss_or_bad_trade", "unclear")

        record = memory.setdefault(
            skey,
            {
                "sample_size": 0,
                "wins": 0,
                "losses": 0,
                "breakevens": 0,
                "avg_r": 0.0,
                "last_seen": args.date,
                "broad_key": bkey,
                "examples": [],
                "decision_quality_counts": defaultdict(int),
            },
        )

        record["sample_size"] += 1
        record["last_seen"] = args.date
        record["examples"].append(trade_id)
        record["decision_quality_counts"][classification] += 1

        r = float(result.get("r_multiple") or 0.0)
        prev_n = max(record["sample_size"] - 1, 0)
        record["avg_r"] = ((record["avg_r"] * prev_n) + r) /
max(record["sample_size"], 1)

        outcome = result.get("result")
        if outcome == "win":
            record["wins"] += 1
        elif outcome == "loss":
            record["losses"] += 1
        elif outcome == "breakeven":
            record["breakevens"] += 1

        if record["wins"] > 0 and record["losses"] > 0:
            contradictions.append(
                {
                    "ticker": ticker,
                    "date": args.date,
                    "trade_id": trade_id,
                    "specific_key": skey,
                    "broad_key": bkey,
                    "memory_update": "Mixed outcomes detected for same/similar
setup. Human review required.",
                    "confidence": "note",
                }
```

```
            )

    # Convert defaultdicts for JSON serialization
    for value in memory.values():
        if isinstance(value.get("decision_quality_counts"), defaultdict):
            value["decision_quality_counts"] =
dict(value["decision_quality_counts"])

    memory_path.write_text(json.dumps(memory, indent=2, ensure_ascii=False) +
"\n", encoding="utf-8")
    with contradiction_path.open("a", encoding="utf-8") as fh:
        for row in contradictions:
            fh.write(json.dumps(row, ensure_ascii=False) + "\n")

    print(json.dumps({"patterns_updated": len(accepted),
"contradictions_logged": len(contradictions)}, indent=2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`backtesting/src/export_to_obsidian.py`

```
from __future__ import annotations

import argparse
import json
from pathlib import Path
from typing import Any

ROOT = Path(__file__).resolve().parents[1]


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Export backtest outputs to
Obsidian markdown.")
    parser.add_argument("--ticker", required=True)
    parser.add_argument("--date", required=True)
    parser.add_argument("--vault-path", default=None, help="Absolute path to
Obsidian vault root.")
    return parser.parse_args()


def read_jsonl(path: Path) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
```

```
    if not path.exists():
        return rows
    with path.open("r", encoding="utf-8") as fh:
        for line in fh:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows


def note_path(vault_root: Path, *parts: str) -> Path:
    path = vault_root.joinpath(*parts)
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def yaml_frontmatter(data: dict[str, Any]) -> str:
    lines = ["---"]
    for key, value in data.items():
        if isinstance(value, list):
            lines.append(f"{key}:")
            for item in value:
                lines.append(f"  - {item}")
        else:
            lines.append(f"{key}: {json.dumps(value) if isinstance(value, str)
else value}")
    lines.append("---")
    return "\n".join(lines)


def write_trade_card(vault_root: Path, ticker: str, trade: dict[str, Any],
result: dict[str, Any]) -> None:
    frontmatter = {
        "trade_id": trade.get("trade_id"),
        "ticker": ticker,
        "date": trade.get("date"),
        "time": trade.get("time"),
        "setup_type": trade.get("setup_type"),
        "direction": trade.get("direction"),
        "primary_timeframe": trade.get("primary_timeframe"),
        "confirmation_timeframe": trade.get("confirmation_timeframe"),
        "result": result.get("result", "unresolved"),
        "r_multiple": result.get("r_multiple"),
        "decision_quality": result.get("valid_loss_or_bad_trade", "unclear"),
        "tags": [
            f"trading/{ticker}",
            f"setup/{trade.get('setup_type', 'unknown')}",
        ],
```

```
    }
    body = f"""# {trade.get('trade_id')}

<!-- AUTO-GENERATED START -->
## Setup
- Level type: {trade.get('level_type')}
- Entry: {trade.get('entry')}
- Stop: {trade.get('stop')}
- Target 1.5R: {trade.get('target_1_5r')}
- Reason codes: {", ".join(trade.get('reason_codes', []))}

## Outcome
- Result: {result.get('result', 'unresolved')}
- R multiple: {result.get('r_multiple')}
- Classification: {result.get('valid_loss_or_bad_trade', 'unclear')}
<!-- AUTO-GENERATED END -->
"""
    path = note_path(vault_root, "05_Trade_Cards", ticker,
f"{trade.get('trade_id')}.md")
    path.write_text(yaml_frontmatter(frontmatter) + "\n\n" + body,
encoding="utf-8")


def write_daily_review(vault_root: Path, ticker: str, date: str, summary:
dict[str, Any]) -> None:
    frontmatter = {
        "ticker": ticker,
        "date": date,
        "accepted_trades": summary.get("total_accepted_trades", 0),
        "tags": [f"trading/{ticker}", "daily-review"],
    }
    body = f"""# {date} {ticker} Review

<!-- AUTO-GENERATED START -->
## Daily Summary
- Day type: {summary.get('day_type')}
- Accepted trades: {summary.get('total_accepted_trades')}
- Wins: {summary.get('wins')}
- Losses: {summary.get('losses')}
- Win rate: {summary.get('win_rate')}
<!-- AUTO-GENERATED END -->
"""
    path = note_path(vault_root, "04_Daily_Reviews", ticker, f"{date} {ticker}
Review.md")
    path.write_text(yaml_frontmatter(frontmatter) + "\n\n" + body,
encoding="utf-8")
```

```
def main() -> int:
    args = parse_args()
    ticker = args.ticker.upper()
    out_dir = ROOT / "outputs" / ticker / args.date
    export_root = Path(args.vault_path) if args.vault_path else ROOT.parent /
"obsidian_vault" / "Trading Mind Vault"

    accepted = read_jsonl(out_dir / "accepted_trades.jsonl")
    results = {row.get("trade_id"): row for row in read_jsonl(out_dir /
"trade_results.jsonl")}
    summary_path = out_dir / "daily_summary.json"
    summary = json.loads(summary_path.read_text(encoding="utf-8")) if
summary_path.exists() else {}

    for trade in accepted:
        write_trade_card(export_root, ticker, trade,
results.get(trade.get("trade_id"), {}))
    write_daily_review(export_root, ticker, args.date, summary)

    print(json.dumps({"trade_cards": len(accepted), "vault_root":
str(export_root)}, indent=2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

The overall recommendation is therefore:

- use TradingView MCP for **chart-native replay extraction, screenshots, and study-aware context**
- always preserve a **fallback provider path**
- keep the event log **append-only JSONL**
- let Python own **resampling, indicators, simulation, stats, and audit**
- let the Ollama worker own **cheap structured tagging only**
- let Opus 4.6 medium own **compressed reasoning and rule review**
- make memory **case-based and human-gated**
- make Obsidian the **searchable semantic vault**
- add Kronos only after the KCU backtesting loop is stable

That architecture is the best fit for your stated goal: a backtesting agent that learns TSLA first, generalizes to new tickers later, stays auditable, stays affordable, and can answer concrete questions like “which setup works best on TSLA from 10–11 AM?” with evidence rather than vibes. [27]

---

[1] [9] [12] [13] [27] https://github.com/tradesdontlie/tradingview-mcp

https://github.com/tradesdontlie/tradingview-mcp

2 19 20 24 https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

3 https://jsonlines.org/
https://jsonlines.org/

4 14 https://docs.alpaca.markets/us/reference/stockbars
https://docs.alpaca.markets/us/reference/stockbars

5 11 https://docs.ollama.com/api/introduction
https://docs.ollama.com/api/introduction

6 23 https://github.com/shiyu-coder/Kronos
https://github.com/shiyu-coder/Kronos

7 8 https://docs.anthropic.com/en/docs/claude-code/overview
https://docs.anthropic.com/en/docs/claude-code/overview

10 21 https://obsidian.md/help/properties
https://obsidian.md/help/properties

15 https://docs.alpaca.markets/us/docs/about-market-data-api
https://docs.alpaca.markets/us/docs/about-market-data-api

16 https://massive.com/docs/
https://massive.com/docs/

17 https://massive.com/docs/rest/stocks/overview
https://massive.com/docs/rest/stocks/overview

18 25 https://interactivebrokers.github.io/tws-api/historical_bars.html
https://interactivebrokers.github.io/tws-api/historical_bars.html

22 https://obsidian.md/help/plugins/search
https://obsidian.md/help/plugins/search

26 https://ojs.aaai.org/index.php/AAAI/article/view/39730/43691
https://ojs.aaai.org/index.php/AAAI/article/view/39730/43691